Para saber mais: operadores

No PHP, nós temos diversos tipos de operadores para lidar com os dados que estamos trabalhando em nossa aplicação. Vou detalhar melhor alguns deles aqui, que já mencionei nos vídeos:

Operadores de atribuição

Os operadores de atribuição são usados para atribuir um valor a uma variável. O operador de atribuição básico é o `=` (sinal de igual). Por exemplo:

```
$valor = 5; // Atribui o valor 5 à variável chamada $valor
```

Existem também operadores de atribuição combinados, que são uma forma abreviada de atribuição. Por exemplo, o operador `+=` adiciona um valor à variável existente. Assim:

```
$valor = 10;
$valor += 15; // Equivalente a $valor = $valor + 15;
```

Operadores aritméticos

Os operadores aritméticos (ou operadores matemáticos) são usados para realizar operações matemáticas básicas. Alguns dos operadores matemáticos no PHP são:

- `+` (adição)
- `-` (subtração)
- `*` (multiplicação)
- `/` (divisão)
- `%` (resto da divisão)
- `**` (potência)

Exemplo:

```
$a = 10 + 5;  // Atribui o valor 15 à variável $a
$b = 10 - 5;  // Atribui o valor 5 à variável $b
$c = 10 * 5;  // Atribui o valor 50 à variável $c
$d = 10 / 5;  // Atribui o valor 2 à variável $d
$e = 10 % 3;  // Atribui o valor 1 à variável $e (o resto da divisão)
$f = 10 ** 3; // Atribui o valor 1000 à variável $f (10 elevado a 3)
```

Operadores de comparação

Os operadores de comparação são usados para comparar valores. Eles retornam um valor booleano (verdadeiro ou falso). Trabalharemos melhor com eles quando estivermos na aula de condicionais, em que vamos modificar o fluxo da aplicação dada alguma condição. Alguns operadores de comparação no PHP são:

- `==` (igual a)
- `===` (idêntico a)
- `!=` (diferente de)
- `<>` (diferente de)
- `!==` (não idêntico a)
- `>` (maior que)
- `>=` (maior ou igual a)
- `<` (menor que)
- `<=` (menor ou igual a)

Exemplo:

```
$a = 10; // Atribui o valor 10 à variável $a
$b = 5;  // Atribui o valor 5 à variável $b
$c = 30; // Atribui o valor 30 à variável $c

$igual = ($b == $a);        // Nesse caso, a variável $igual ficará com o valor f
$diferente = ($b != $c);    // A variável $diferente ficará com o valor true
$maior = ($b > $a);        // A variável maior ficará com o valor false
$menorIgual = ($b <= $c);  // A variável $menorIgual ficará com o valor true
```

Diferença entre `==` e `===`

A diferença entre `==` e `===` (e entre `!=` e `!==`) é que os tipos são levados em consideração no operador de identidade. Por exemplo:

```
$a = "1"; // String
$b = 1;   // Inteiro

$igual = ($a == $b);       // true, pois os valores são iguais
$identico = ($a === $b);   // false, pois os tipos são diferentes (string vs int)
$diferente = ($a != $b);   // false, pois os valores NÃO são diferentes
$naoIdentico = ($a !== $b); // true, pois eles não são idênticos em tipo
```

Operadores lógicos

Esses operadores são usados quando queremos verificar duas ou mais condições e/ou expressões na aplicação. Eles fazem a comparação de valores booleanos e retornam também um resultado booleano.

Os principais e mais comuns operadores lógicos no PHP são: **AND** (`&&`), **OR** (`||`) e **NOT** (`!`).

- **AND (`&&`):** É usado para verificar se duas condições são verdadeiras. Se ambas forem verdadeiras, o resultado será verdadeiro.

```
$a = true;
$b = false;
$comparacao = $a && $b; // O resultado será false
```

- **OR (`||`):** É usado para verificar se pelo menos uma das condições é verdadeira. Se pelo menos uma for verdadeira, o resultado será verdadeiro.

```
$a = true;
$b = false;
$comparacao = $a || $b; // O resultado será true
```

- **NOT (`!`):** É usado para negar uma condição. Se a condição for verdadeira, o resultado será falso (e vice-versa).

```
$a = true;
$negacao = !$a; // O resultado será false
```

Operadores de incremento e decremento

O operador de incremento é usado para aumentar o valor de uma variável em 1. Existem dois tipos: o de **pré-incremento** (`++$variavel`) e o de **pós-incremento** (`$variavel++`).

- **Pré-incremento (`++$num`):** Aumenta o valor antes de usar a variável na expressão.

```
$num = 5;
$resultado = ++$num; // $num vai para 6 e depois é atribuído
echo $num;       // Exibe 6
echo $resultado; // Exibe 6
```

- **Pós-incremento (`$num++`):** Aumenta o valor depois de usar a variável na expressão.

```
$num = 5;
$resultado = $num++; // $num (5) é atribuído primeiro, depois incrementado
echo $num;       // Exibe 6
echo $resultado; // Exibe 5
```

O mesmo princípio se aplica aos operadores de **decremento**: pré-decremento (`--$variavel`) e pós-decremento (`$variavel--`).

Outros operadores

O PHP ainda possui outros operadores, como operadores de operação bit a bit (para lidar com operações em números binários), operadores de texto (concatenação), etc. O que apresentamos já é uma boa introdução aos operadores do PHP e nós podemos aprender os demais conforme a necessidade surgir.

Para saber mais: operador ternário

Além das estruturas de controle de fluxo que vimos até aqui (`if/elseif/else` , `switch case` e `match`), o PHP também possui um operador que nos permite tomar decisões que resultam em uma expressão: o **operador ternário** `?:` .

Esse operador serve para avaliar determinada expressão e:

1. Caso seja **verdadeira**, retornar uma segunda expressão.
2. Caso seja **falsa**, retornar uma terceira expressão.

O uso de 3 possíveis expressões (a condição, o resultado positivo e o resultado negativo) é o que dá nome a esse operador.

Sintaxe

A sintaxe básica é a seguinte:

```
$resultado = $expressao1 ? $expressao2 : $expressao3;
```

Como funciona:

- Se `$expressao1` for verdadeira (**true**), `$resultado` terá o valor de `$expressao2` .
- Caso contrário (se for **false**), `$resultado` terá o valor de `$expressao3` .

Exemplo Prático

Imagine que você quer verificar se um usuário pode votar com base na sua idade:

```
<?php

$idade = 20;

// Utilizando o operador ternário
$podeVotar = ($idade >= 18) ? "Sim, pode votar" : "Não pode votar";

echo $podeVotar; // Exibe: Sim, pode votar
```

Neste exemplo:

- **Condição:** `$idade >= 18`
- **Se verdadeiro:** retorna `"Sim, pode votar"`
- **Se falso:** retorna `"Não pode votar"`

Conclusão

Esse operador não é tão simples de se entender no início, então não se preocupe se não tiver ficado totalmente claro por enquanto. Conforme você for avançando nos estudos, esse tipo de operador mais "compacto" fará mais sentido e ajudará a deixar o seu código mais limpo quando precisar de atribuições simples baseadas em condições.

Para saber mais: Switch Case

Uma alternativa ao `if/else` é o `switch case`, que é uma estrutura de controle de fluxo que permite executar diferentes ações com base no valor de uma expressão. É uma forma mais simplificada e legível de escrever vários blocos `if/else` encadeados, desde que as operações sejam somente de igualdade.

Sintaxe do Switch Case

A sintaxe do `switch case` em PHP é a seguinte:

```
switch (expressão) {
    case valor1:
        // código a ser executado se a expressão for igual a valor1
        break;
    case valor2:
        // código a ser executado se a expressão for igual a valor2
        break;
    case valor3:
        // código a ser executado se a expressão for igual a valor3
        break;
    default:
        // código a ser executado se nenhum dos casos anteriores for atendido
        break;
}
```

Como funciona:

1. **Expressão**: É uma variável ou uma expressão de código que será avaliada.
2. **Case**: Cada `case` é uma possível condição que pode ser atendida pela expressão. Quando a expressão é igual ao valor especificado, o código correspondente é executado.
3. **Break**: A palavra-chave `break` é usada para sair do `switch case` após a execução do código. Sem o `break`, o PHP continuaria executando os casos seguintes mesmo que não houvesse correspondência.
4. **Default**: O `case default` é opcional e funciona como um "senão", sendo executado quando nenhum dos casos especificados é atendido.

Exemplo Prático

Veja um exemplo simples de uso do `switch case` em PHP para verificar o dia da semana com base em um número inteiro:

```
<?php

$dia = 3;

switch ($dia) {
```

```
    case 1:
        $nomeDia = "domingo";
        break;
    case 2:
        $nomeDia = "segunda-feira";
        break;
    case 3:
        $nomeDia = "terça-feira";
        break;
    case 4:
        $nomeDia = "quarta-feira";
        break;
    case 5:
        $nomeDia = "quinta-feira";
        break;
    case 6:
        $nomeDia = "sexta-feira";
        break;
    case 7:
        $nomeDia = "sábado";
        break;
    default:
        $nomeDia = "Dia inválido";
        break;
}

echo "O dia $dia é $nomeDia";
```

Nesse exemplo, a expressão é a variável `$dia`, que contém o valor `3`. O `switch case` verifica o valor e executa o código correspondente ao caso `3`. O resultado impresso será: "O dia 3 é terça-feira".

Conclusão

A principal vantagem de usar o `switch case` é tornar o código mais fácil de entender e mais legível em comparação ao `if/else`, especialmente quando existem várias condições possíveis baseadas em uma única variável.